

Experiment 12: Iris Flower Classification using KNN

Aim

To implement **Iris Flower Classification** using the **K-Nearest Neighbours (KNN)** algorithm.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

print("Predicted Values:")
print(y_pred)
```

```

print("\nActual Values:")
print(y_test)

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Predict new flower sample
new_sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_sample)

print("\nNew Flower Prediction:")
print(iris.target_names[prediction[0]])

```

Sample Output

```

... Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Actual Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy:
1.0

Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

Classification Report:
              precision    recall  f1-score   support

   setosa               1.00      1.00      1.00        19
  versicolor            1.00      1.00      1.00        13
   virginica            1.00      1.00      1.00        13

   accuracy                1.00          45
  macro avg               1.00      1.00      1.00          45
 weighted avg               1.00      1.00      1.00          45

New Flower Prediction:
setosa

```